

Vertex buffer creation

- [Introduction](#)
- [Buffer creation](#)
- [Memory requirements](#)
- [Memory allocation](#)
- [Filling the vertex buffer](#)
- [Binding the vertex buffer](#)

Introduction

Buffers in Vulkan are regions of memory used for storing arbitrary data that can be read by the graphics card. They can be used to store vertex data, which we'll do in this chapter, but they can also be used for many other purposes that we'll explore in future chapters. Unlike the Vulkan objects we've been dealing with so far, buffers do not automatically allocate memory for themselves. The work from the previous chapters has shown that the Vulkan API puts the programmer in control of almost everything and memory management is one of those things.

Buffer creation

Create a new function `createVertexBuffer` and call it from `initVulkan` right before `createCommandBuffers`.

```
void initVulkan() {
    createInstance();
    setupDebugMessenger();
    createSurface();
    pickPhysicalDevice();
    createLogicalDevice();
    createSwapChain();
    createImageViews();
    createRenderPass();
    createGraphicsPipeline();
    createFramebuffers();
    createCommandPool();
    createVertexBuffer();
    createCommandBuffers();
    createSyncObjects();
}

...

void createVertexBuffer() {
```

Creating a buffer requires us to fill a `VkBufferCreateInfo` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkBufferCreateInfo.html>) structure.

```
VkBufferCreateInfo bufferInfo{};
bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
bufferInfo.size = sizeof(vertices[0]) * vertices.size();
```

The first field of the struct is `size`, which specifies the size of the buffer in bytes. Calculating the byte size of the vertex data is straightforward with `sizeof`.

```
bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;
```

The second field is `usage`, which indicates for which purposes the data in the buffer is going to be used. It is possible to specify multiple purposes using a bitwise or. Our use case will be a vertex buffer, we'll look at other types of usage in future chapters.

```
bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
```

Just like the images in the swap chain, buffers can also be owned by a specific queue family or be shared between multiple at the same time. The buffer will only be used from the graphics queue, so we can stick to exclusive access.

The `flags` parameter is used to configure sparse buffer memory, which is not relevant right now. We'll leave it at the default value of `0`.

We can now create the buffer with `vkCreateBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCreateBuffer.html>). Define a class member to hold the buffer handle and call it `vertexBuffer`.

```
VkBuffer vertexBuffer;

...

void createVertexBuffer() {
    VkBufferCreateInfo bufferInfo{};
    bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
    bufferInfo.size = sizeof(vertices[0]) * vertices.size();
    bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;
    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;

    if (vkCreateBuffer(device, &bufferInfo, nullptr, &vertexBuffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to create vertex buffer!");
    }
}
```

The buffer should be available for use in rendering commands until the end of the program and it does not depend on the swap chain, so we'll clean it up in the original `cleanup` function:

```
void cleanup() {  
    cleanupSwapChain();  
  
    vkDestroyBuffer(device, vertexBuffer, nullptr);  
  
    ...  
}
```

Memory requirements

The buffer has been created, but it doesn't actually have any memory assigned to it yet. The first step of allocating memory for the buffer is to query its memory requirements using the aptly named

`vkGetBufferMemoryRequirements` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkGetBufferMemoryRequirements.html>) function.

```
VkMemoryRequirements memRequirements;  
vkGetBufferMemoryRequirements(device, vertexBuffer, &memRequirements);
```

The `VkMemoryRequirements` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkMemoryRequirements.html>) struct has three fields:

- `size`: The size of the required amount of memory in bytes, may differ from `bufferInfo.size`.
- `alignment`: The offset in bytes where the buffer begins in the allocated region of memory, depends on `bufferInfo.usage` and `bufferInfo.flags`.
- `memoryTypeBits`: Bit field of the memory types that are suitable for the buffer.

Graphics cards can offer different types of memory to allocate from. Each type of memory varies in terms of allowed operations and performance characteristics. We need to combine the requirements of the buffer and our own application requirements to find the right type of memory to use. Let's create a new function `findMemoryType` for this purpose.

```
uint32_t findMemoryType(uint32_t typeFilter, VkMemoryPropertyFlags properties) {  
  
}
```

First we need to query info about the available types of memory using

`vkGetPhysicalDeviceMemoryProperties` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkGetPhysicalDeviceMemoryProperties.html>).

```
VkPhysicalDeviceMemoryProperties memProperties;  
vkGetPhysicalDeviceMemoryProperties(physicalDevice, &memProperties);
```

The `VkPhysicalDeviceMemoryProperties` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkPhysicalDeviceMemoryProperties.html>) structure has two arrays `memoryTypes` and `memoryHeaps`. Memory heaps are distinct memory resources like dedicated VRAM and swap space in RAM for when VRAM runs out. The different types of memory exist within these heaps. Right now we'll only concern

ourselves with the type of memory and not the heap it comes from, but you can imagine that this can affect performance.

Let's first find a memory type that is suitable for the buffer itself:

```
for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
    if (typeFilter & (1 << i)) {
        return i;
    }
}

throw std::runtime_error("failed to find suitable memory type!");
```

The `typeFilter` parameter will be used to specify the bit field of memory types that are suitable. That means that we can find the index of a suitable memory type by simply iterating over them and checking if the corresponding bit is set to `1`.

However, we're not just interested in a memory type that is suitable for the vertex buffer. We also need to be able to write our vertex data to that memory. The `memoryTypes` array consists of `VkMemoryType` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkMemoryType.html>) structs that specify the heap and properties of each type of memory. The properties define special features of the memory, like being able to map it so we can write to it from the CPU. This property is indicated with `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`, but we also need to use the `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` property. We'll see why when we map the memory.

We can now modify the loop to also check for the support of this property:

```
for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
    if ((typeFilter & (1 << i)) && (memProperties.memoryTypes[i].propertyFlags & proper
ties) == properties) {
        return i;
    }
}
```

We may have more than one desirable property, so we should check if the result of the bitwise AND is not just non-zero, but equal to the desired properties bit field. If there is a memory type suitable for the buffer that also has all of the properties we need, then we return its index, otherwise we throw an exception.

Memory allocation

We now have a way to determine the right memory type, so we can actually allocate the memory by filling in the `VkMemoryAllocateInfo` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkMemoryAllocateInfo.html>) structure.

```
VkMemoryAllocateInfo allocInfo{};
allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
allocInfo.allocationSize = memRequirements.size;
allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, VK_MEMORY_PR
OPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT);
```

Memory allocation is now as simple as specifying the size and type, both of which are derived from the memory requirements of the vertex buffer and the desired property. Create a class member to store the handle to the memory and allocate it with `vkAllocateMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkAllocateMemory.html>).

```
VkBuffer vertexBuffer;
VkDeviceMemory vertexBufferMemory;

...

if (vkAllocateMemory(device, &allocInfo, nullptr, &vertexBufferMemory) != VK_SUCCESS) {
    throw std::runtime_error("failed to allocate vertex buffer memory!");
}
```

If memory allocation was successful, then we can now associate this memory with the buffer using `vkBindBufferMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkBindBufferMemory.html>):

```
vkBindBufferMemory(device, vertexBuffer, vertexBufferMemory, 0);
```

The first three parameters are self-explanatory and the fourth parameter is the offset within the region of memory. Since this memory is allocated specifically for this the vertex buffer, the offset is simply `0`. If the offset is non-zero, then it is required to be divisible by `memRequirements.alignment`.

Of course, just like dynamic memory allocation in C++, the memory should be freed at some point. Memory that is bound to a buffer object may be freed once the buffer is no longer used, so let's free it after the buffer has been destroyed:

```
void cleanup() {
    cleanupSwapChain();

    vkDestroyBuffer(device, vertexBuffer, nullptr);
    vkFreeMemory(device, vertexBufferMemory, nullptr);
}
```

Filling the vertex buffer

It is now time to copy the vertex data to the buffer. This is done by mapping the buffer memory (https://en.wikipedia.org/wiki/Memory-mapped_I/O) into CPU accessible memory with `vkMapMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkMapMemory.html>).

```
void* data;
vkMapMemory(device, vertexBufferMemory, 0, bufferInfo.size, 0, &data);
```

This function allows us to access a region of the specified memory resource defined by an offset and size. The offset and size here are `0` and `bufferInfo.size`, respectively. It is also possible to specify the special value `VK_WHOLE_SIZE` to map all of the memory. The second to last parameter can be used to specify flags, but there aren't any available yet in the current API. It must be set to the value `0`. The last parameter specifies the output for the pointer to the mapped memory.

```
void* data;
vkMapMemory(device, vertexBufferMemory, 0, bufferInfo.size, 0, &data);
memcpy(data, vertices.data(), (size_t) bufferInfo.size);
vkUnmapMemory(device, vertexBufferMemory);
```

You can now simply `memcpy` the vertex data to the mapped memory and unmap it again using `vkUnmapMemory` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkUnmapMemory.html>). Unfortunately the driver may not immediately copy the data into the buffer memory, for example because of caching. It is also possible that writes to the buffer are not visible in the mapped memory yet. There are two ways to deal with that problem:

- Use a memory heap that is host coherent, indicated with `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT`
- Call `vkFlushMappedMemoryRanges` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkFlushMappedMemoryRanges.html>) after writing to the mapped memory, and call `vkInvalidateMappedMemoryRanges` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkInvalidateMappedMemoryRanges.html>) before reading from the mapped memory

We went for the first approach, which ensures that the mapped memory always matches the contents of the allocated memory. Do keep in mind that this may lead to slightly worse performance than explicit flushing, but we'll see why that doesn't matter in the next chapter.

Flushing memory ranges or using a coherent memory heap means that the driver will be aware of our writes to the buffer, but it doesn't mean that they are actually visible on the GPU yet. The transfer of data to the GPU is an operation that happens in the background and the specification simply tells us (<https://www.khronos.org/registry/vulkan/specs/1.3-extensions/html/chap7.html#synchronization-submission-host-writes>) that it is guaranteed to be complete as of the next call to `vkQueueSubmit` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkQueueSubmit.html>).

Binding the vertex buffer

All that remains now is binding the vertex buffer during rendering operations. We're going to extend the `recordCommandBuffer` function to do that.

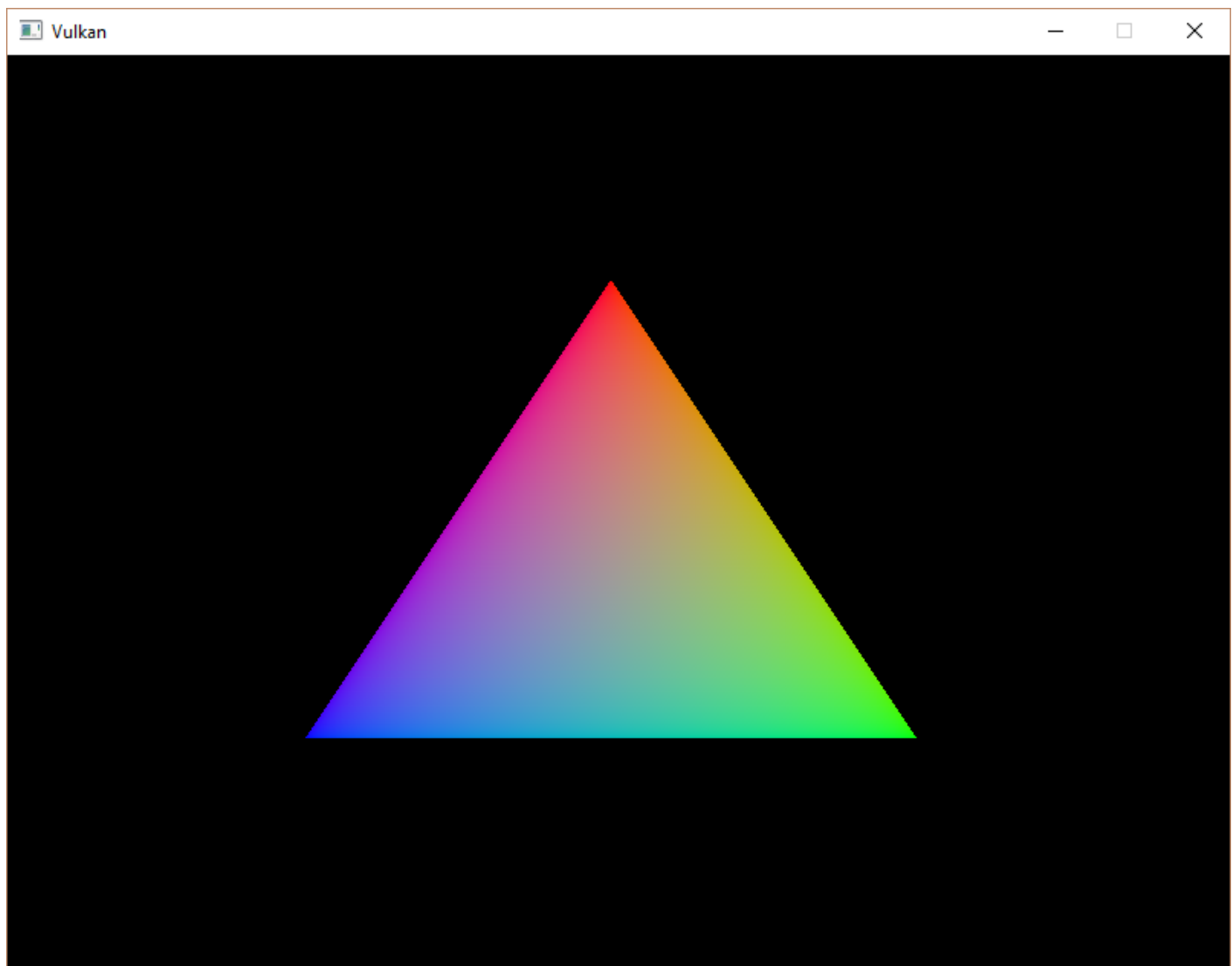
```
vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);

VkBuffer vertexBuffers[] = {vertexBuffer};
VkDeviceSize offsets[] = {0};
vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);

vkCmdDraw(commandBuffer, static_cast<uint32_t>(vertices.size()), 1, 0, 0);
```

The `vkCmdBindVertexBuffers` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdBindVertexBuffers.html>) function is used to bind vertex buffers to bindings, like the one we set up in the previous chapter. The first two parameters, besides the command buffer, specify the offset and number of bindings we're going to specify vertex buffers for. The last two parameters specify the array of vertex buffers to bind and the byte offsets to start reading vertex data from. You should also change the call to `vkCmdDraw` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdDraw.html>) to pass the number of vertices in the buffer as opposed to the hardcoded number `3`.

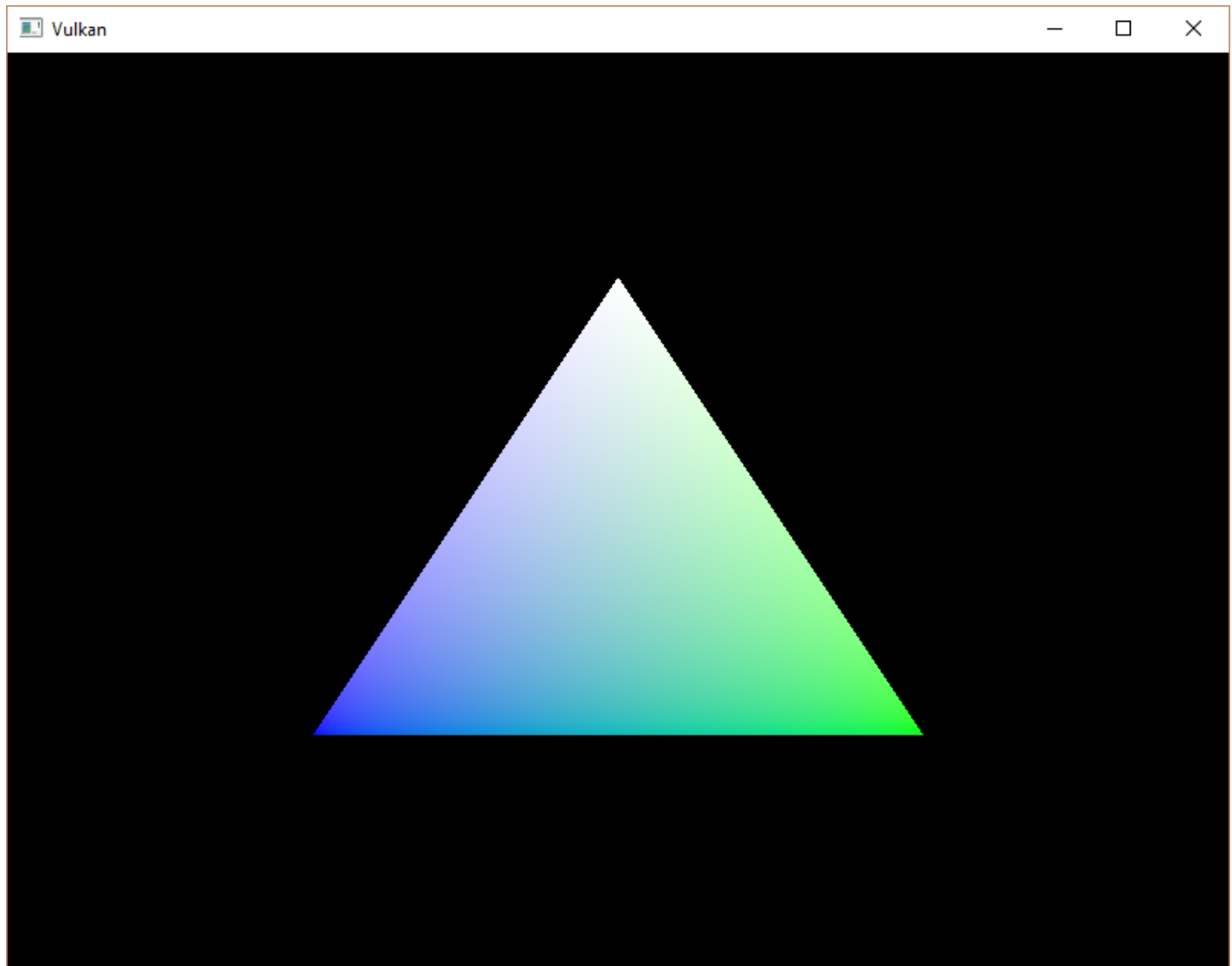
Now run the program and you should see the familiar triangle again:



Try changing the color of the top vertex to white by modifying the `vertices` array:

```
const std::vector<Vertex> vertices = {  
    {{0.0f, -0.5f}, {1.0f, 1.0f, 1.0f}},  
    {{0.5f, 0.5f}, {0.0f, 1.0f, 0.0f}},  
    {{-0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}}  
};
```

Run the program again and you should see the following:



In the next chapter we'll look at a different way to copy vertex data to a vertex buffer that results in better performance, but takes some more work.

[C++ code \(../code/19_vertex_buffer.cpp\)](#) / [Vertex shader \(../code/18_shader_vertexbuffer.vert\)](#) / [Fragment shader \(../code/18_shader_vertexbuffer.frag\)](#)

[Previous \(/en/Vertex_buffers/Vertex_input_description\)](/en/Vertex_buffers/Vertex_input_description)

[Next \(/en/Vertex_buffers/Staging_buffer\)](/en/Vertex_buffers/Staging_buffer)